UNIVERSIDAD FASTA FACULTAD DE INGENIERÍA

Carrera: Ingeniería en Informática
Asignatura: Programación A

Capítulo 2: Estructura secuencial

- Lenguajes de programación: Pascal
- Estructura de un programa (Declaraciones y Sentencias ejecutables)
- Operadores aritméticos
- Errores sintácticos y semánticos
- Sentencias estructura secuencial

Versión 1 – 03/2023

Un lenguaje algorítmico
- ✓ tiene una **sintaxis** y un **vocabulario** limitado
- ✓ comprende sólo el tipo de **acciones** que una **computadora** puede entender y realizar
- ✓ cada acción tiene una **forma rigurosa** de expresarse y una **significación** propia (**sintaxis y semántica**)

La descripción de un **algoritmo** en un **lenguaje de programación** constituye un **programa**

Existen muchos **lenguajes de programación**, con diferentes características y que responden a diferentes **paradigmas** de programación. De forma genérica, un paradigma es una **forma de ver**, **entender** y **representar** el mundo.
Un paradigma de programación es una colección de **conceptos** que **guían** el proceso de **construcción** de un sistema, determinando su **estructura.** Define la **estrategia** a utilizar para atacar la complejidad, controlando la **forma de pensar** y formular los programas.
Entre los paradigmas de programación existe el **declarativo**, dentro del cual se encuentran **la programación lógica** (basada en principios de la lógica formal que expresan los problemas como teorías, o sea hechos más reglas) y la **programación funcional** (con evaluación de funciones matemáticas).

Buscar lenguajes de programación que sigan el paradigma de programación declarativo

Gran parte de los lenguajes responden al **paradigma imperativo**, en el cual las sentencias del programa son **órdenes** o instrucciones que se le dan a la computadora. Dentro de este paradigma se encuentran el **procedural**, y el **orientado a objetos**.

La programación procedural busca particionar un programa en múltiples tareas con funciones bien definidas ("**divide y vencerás**") en pos de facilitar su entendimiento. La **programación estructurada** extiende a la procedural, agregando **restricciones** que **faciliten** su posterior modificación y **mantenimiento** (por ejemplo: evitar variables globales, evitar salidas abruptas de ciclos y subrutinas).

Además de los lenguajes de programación, existen **otras maneras** de **especificar** un **algoritmo** ya sea de manera **gráfica** (por ejemplo, el **diagrama de flujo**) o de manera **escrita** coloquial, simplificando la sintaxis (**pseudocódigo**). Esto permite expresar o aprender un algoritmo sin necesidad de abordar las **dificultades sintácticas** propias de cada lenguaje. Históricamente, estas especificaciones han sido utilizadas para la **enseñanza** inicial de la algoritmia.

En esta asignatura se utilizará **Pascal**, un lenguaje imperativo que permite aplicar los principios de la **programación estructurada**. Es adecuado para la enseñanza inicial de la programación, ya que resulta **más amigable** que otros lenguajes que son más utilizados en el mercado.

Buscar otros lenguajes de programación que respondan al paradigma de programación imperativo

Un **programa** en **Pascal** se compone de:
- ✓ Sección **declarativa**: donde se describen características de los **objetos** que utiliza el programa.
- ✓ Sección **ejecutable**: donde se describen las acciones que el programa realizará, estas **sentencias** pueden ser simples o compuestas.

Simples
No contienen otra sentencia

Ejemplos : `Write, Read`

Compuestas
Grupo de sentencias simples encerradas entre un BEGIN y un END.

```
Ejemplo: BEGIN
           sentencia 1;
           sentencia 2;

           sentencia n;
         END
```

Un programa en Pascal consta de un **encabezamiento** y un bloque dividido en secciones; las primeras son **declarativas**, y la última **ejecutable**. Todas las secciones excepto la última pueden estar vacías.
El siguiente cuadro presenta la **estructura básica**, luego se incorporarán otras secciones.

`PROGRAM <identificador>;  {encabezamiento del programa}`
 	
`CONST`
`<definiciones de constantes>;`
 	
`VAR`
`<declaración de variables>;`
	DECLARATIVA

`BEGIN`
`<sentencias>`
`END.`
	EJECUTABLE

VARIABLES. CONSTANTES

En una **función matemática** ya es posible distinguir elementos **variables** y **constantes**. Por ejemplo, la superficie del círculo está dada por "PI por radio al cuadrado". En esta fórmula, PI es constante y el radio es variable.

Una **variable** es un objeto de un programa cuyo valor puede "variar", es una **celda** de **memoria** donde se **almacena información**.

Tiene dos atributos o características propias: un **identificador** y un **tipo**, además de su **valor** o contenido, y la **dirección** de memoria que ocupa.

Es importante que el **identificador** o nombre de una variable informe acerca de su contenido ("**representativo**"), esto hace más fácil la **comprensión** de un algoritmo. Se deben establecer criterios para la elección de los identificadores, se adoptará que las palabras que lo integran comiencen con mayúscula. Ej: `SalarioBruto`

El **tipo de dato** de una variable está dado por la **naturaleza** del valor que almacena, por ejemplo, una edad es un valor numérico **entero** positivo, un precio es **real** (numérico con decimales), un nombre es una **cadena** de caracteres, etc.

En relación a esto, otra característica que clasifica a los lenguajes es si son **tipados** o no **tipados**. Pascal es un lenguaje **fuertemente tipado**, todas las variables que se utilizan en un programa, deben ser **declaradas** especificando su **nombre** y **tipo**.

Una **constante** es un objeto de un programa cuyo **valor no cambia**.

Ej: `57`, `-32.79`

Pueden tener un **identificador asociado** y se declaran en la sección

```
Const
    Porcentaje = 15;
```

Cada vez que aparezca Porcentaje en el código se sustituye por 15. La ventaja es que si se cambia el valor de la constante, por ejemplo `Porcentaje = 23`, el código se actualiza automáticamente en todas sus ocurrencias.

Tipos de Datos

El lenguaje provee **tipos** estándar o **simples** para **enteros**, **reales**, **caracteres** y **lógicos** (verdadero y falso). Los dos primeros tienen a su vez varias posibilidades de **rango** y **precisión**. El programador puede declarar otros **tipos propios** para describir sus datos.

Se analizarán en una primera etapa los tipos numéricos entero y real, los otros tipos se verán más adelante.

REALES	*SINGLE*	*1.5E-45 a 3.4E38*
	REAL	*2.9E-39 a 1.7E38*
	DOUBLE	*5.0E-324 a 1.7E308*
	EXTENDED	*1.9E-4851 a 1.1E4932*
ENTEROS	*SHORTINT*	*-128 a 127*
	BYTE	***0 a 255***
	WORD	***0 a 65535***
	INTEGER	***-32,768 a 32,767***
	LONGINT	*-2,147,483,648 a 2,147,483,648*

Pascal - Otros elementos del lenguaje y características generales

Delimitadores: se utilizan para agrupar o separar declaraciones o sentencias.
Ejemplos: `;` - `BEGIN` - `END`

Comentarios: se utilizan para hacer **aclaraciones** en cualquier lugar del programa (son **ignorados** por el **compilador**). Se recomienda su uso para **mayor legibilidad**. Se encierran entre llaves o paréntesis y asterisco. Ejemplo: `{esto es un comentario}`, `(*esto es un comentario*)`. Evitar comentarios que repitan lo que realiza claramente el código.

Palabras reservadas: son aquellas que sólo pueden usarse para un **fin específico**.
Ejemplos: `program`, `begin`, etc.

Identificador: es el nombre que se le asigna a un objeto (programa, constante, variable), debe comenzar con una letra y a continuación letras o dígitos o el caracter '_'.
Ej: `Program Determina_Triangulo`

En Pascal, **no** hay **distinción** entre **mayúsculas y minúsculas**. Otros lenguajes, denominados *case sensitive* diferencian mayúsculas de minúsculas ()
Ej: `Determina_Triangulo` = `determina_TRIANGULO`

No hay restricción respecto a la longitud, pero debe considerarse la **legibilidad** así como también la facilidad para comprender y escribir el código. Es conveniente que el **identificador** sea **representativo**, o sea que esté asociado al rol de dicho objeto. Por ejemplo, no es recomendable usar nombres como `var1`, `var2`, `var3`, etc. ya que resultan difíciles de recordar, seguir y pueden generar confusiones.

Sección declarativa

Lo visto hasta ahora permite codificar la sección declarativa de un programa, de acuerdo a la estructura presentada anteriormente se especifican las secciones como sigue:

```
CONST  {opcional, se declara las constantes, identificándolas con un nombre}
        id1 = v1;  {id1, id2 -> identificadores}
        id2 = v2;  {v1, v2 -> valores}
```

…….

```
VAR {obligatorio, declarar todas las variables que utiliza el programa}.
       id1 : tipo1;
       id2 : tipo2;
       id3, id4 : tipo3; {cuando distintas variables tienen el mismo tipo}
```

Debe especificarse el nombre de la variable y su tipo, donde el nombre es un identificador y el tipo puede ser estándar o declarado previamente por el programador.

Resolver el ítem 1 de los ejercicios propuestos al final del capítulo.

OPERADORES ARITMÉTICOS. EXPRESIONES

Cada tipo de dato, tiene sus propios operadores con los cuales se construyen **expresiones**. Los datos numéricos pueden operarse aritméticamente con los operadores que se detallan a continuación. Existen también los operadores **relacionales** y los **lógicos**. Los relacionales permiten **comparar** datos del mismo tipo, generando un resultado lógico (verdadero o falso). Los lógicos permitirán combinar expresiones lógicas. Ambos serán profundizados posteriormente.

Tomando las siguientes variables:

```
L, N, M, P: integer;
A, B, E: real;
```

OPERADORES ARITMETICOS	TIPOS de OPERANDOS	TIPO de RESULTADO	EJEMPLOS de EXPRESIONES
* , / , + , -	entero, real	entero, real	`A * B / (6 - L),  N + 5`
MOD (resto entero)	entero	entero	`P MOD 2 , M DIV N`

Orden de prioridad:

1. * , / , MOD
2. + , -

- Dicha **prioridad** se altera utilizando **paréntesis**
- Si en una expresión el orden de precedencia de los operadores es el **mismo**, se resuelve de **izquierda a derecha**.

Evaluar si las siguientes expresiones son correctas, y el orden de evaluación dentro de cada una: A MOD P ; L * 100 / (N – M) ; A + B / (E * 100)

Existen **funciones definidas** por el lenguaje que facilitan los cálculos. Por ejemplo, si se desea obtener la raíz cuadrada un valor X, se invoca la función `Sqrt(X)`.

Al final del capítulo se presenta una tabla de las funciones más utilizadas en Pascal.

Sección ejecutable

ESTRUCTURA SECUENCIAL. SENTENCIAS

Una variable recibe valor de dos formas: **asignación** y **lectura**, en ambos casos si tiene un valor almacenado previamente lo **pierde**. Su contenido se puede visualizar mediante **escritura**

Acción → Asignación
Acción → Lectura
Acción → Escritura

Estas son las acciones básicas que constituyen los "pasos de un algoritmo". Las **sentencias** o **instrucciones** de un programa desencadenan las respectivas **acciones**.

- **ASIGNACION**

Es una operación que permite almacenar un valor en una variable

```
VARIABLE := <expresión>;
```

Ejemplos:

```
A := 3;
B := A * 10;
C := A + B;
```

El **valor** o **expresión** situado a la **derecha** se **almacena** en la **variable** de la **izquierda**, o sea que se está almacenando un dato en la posición de memoria asociada al nombre de esa variable.

El **tipo** de la **expresión** debe ser igual al de la **variable**, salvo que:
- La variable sea real y la expresión sea entera

La **variable** de la **izquierda pierde** el **valor** almacenado previamente, si es que lo tenía, al recibir otro por asignación.
Las variables que figuran a la derecha de la asignación no pierden su valor.

Evaluar si las siguientes asignaciones son correctas sintácticamente:
A + 3 <- B ; 3 <- A + B ; B <- B ; B <- A;

- **LECTURA**

```
Readln (lista de variables separadas por comas)
```

Ejemplos:

```
Readln(Jugadores);
Readln(Largo, Ancho);
```

Lee los datos del teclado almacenando en la lista de variables según el **orden** en que se ingresan
Los datos que alimentan las variables de la lista, son ingresados por teclado separados por espacios.

No tiene sentido asignar un valor a una variable inmediatamente antes o después de haber leído sobre la misma, ya que lo pierde quedando el correspondiente a la última operación

Evaluar si las siguientes lecturas son correctas sintácticamente:
Readln(A, C, B); Readln(A, 3); Readln(' H', ola);

- **ESCRITURA**

```
Write(lista de salida);
```

{escribe los elementos de la lista en la pantalla y queda posicionado en la misma línea}

```
Writeln(lista de salida);
```

{escribe los elementos de la lista y queda posicionado al comienzo de la línea siguiente}

Ejemplos: `Write('Ingrese largo y ancho de un rectángulo: ');`

```
Writeln('Cada jugador recibe ', Tocan, ' cartas');
```

Corregir las siguientes instrucciones de escritura:

```
Writeln(el resultado es: Resultado) ;
Writeln('La suma de A + B es A+B');
Writeln('El producto es: , A*B, y la resta es: ', (A-B));
```

Escritura con formato

Es posible establecer, para cada elemento de la lista de salida, el **ancho** del campo donde se escribe. Basta colocar después del elemento el **caracter** **':'** y la cantidad de columnas del campo de escritura.

`Writeln(elemento:Ancho);`

Ancho es un entero que indica la **cantidad de columnas** del campo en el que se escribe el elemento. Puede utilizarse para enteros, cadena, caracteres.

Para los **números reales** se debe especificar la **cantidad** de **decimales** deseada.

`Writeln(elementoReal:Ancho:Decimales);`

Ancho es un entero que especifica la **cantidad total** de dígitos del número real (contando el signo, parte entera, **punto decimal** y dígitos decimales)

Decimales es un entero que especifica la **cantidad** de dígitos **decimales** con los que se escribirá el número real.

Ejemplos:

```
A: real;   C: integer;

A := 3.456;
C := 10;
Write(A:8:2);
Write(C:10);
Write(A:2:4);
```

S A L I D A

C O L U M N A S

1	2	3	4	5	6	7	8	9	10
				3	.	4	6		
								1	0
3	.	4	5	6	0				

Si **no se especifica** formato en los números reales, se visualizan en **notación científica**.

Resolver los ítems 2 y 3 de los ejercicios propuestos al final del capítulo.

ERRORES DE PROGRAMA SINTÁCTICOS Y SEMÁNTICOS

- **Sintácticos**: provienen de no respetar en la **escritura** del código, la sintaxis del lenguaje, por ejemplo:
  `Writeln(A; es el resultado),`

- **Semánticos**: provienen de la **lógica** de la solución, deben ser detectados por el **programador** a través de pruebas con diferentes lotes de datos.

Ejemplo, si se desea promediar tres notas con la siguiente fórmula:

```
N1 + N2 + N3 / 3
```

el resultado es incorrecto, porque suma a N1 y N2 el valor N3 / 3. **Para obtener el promedio, primero deben sumarse las 3 notas y luego dividirlas por 3:**

```
(N1 + N2 + N3) / 3
```

Pensar otros errores y clasificar en sintácticos y semánticos

Ejemplos de programas Pascal. Determinar qué problemática resuelven.

```
Program Uno;
Var
   Largo, Ancho, Diag, Sup: real;
Begin
   Writeln('Ingrese largo y ancho de un rectángulo');
   Readln(Largo, Ancho);
   Sup := Largo * Ancho;
   Writeln('la superficie es ', Sup);
   Diag := sqrt( sqr(Largo) + sqr(Ancho));
   Writeln('la diagonal es ', Diag);
End.

Program Dos;
Const
    Mazo = 40;
Var
    Jugadores, Restan, Tocan: byte;
Begin
   Writeln('Ingrese cant. de jugadores: ');  Readln(Jugadores);
   Tocan := Mazo DIV Jugadores;
   Restan := Mazo MOD Jugadores;
   Writeln('Cada jugador recibe ', Tocan, ' cartas');
   Writeln('Sobran ', Restan, ' cartas');
End.
```

Resolver los ítems 4 y 5 de los ejercicios propuestos al final del capítulo.

- **Algunas funciones aritméticas**

Nombre	Tipo del argumento	Resultado	Tipo del resultado
`Abs(x)`	Real o Integer	el valor absoluto del argumento	Real o Integer
`Frac(x)`	Real	la parte decimal del argumento	Real
`Int(x)`	Real	la parte entera del argumento	Real
`Round(x)`	Real o Integer	el entero más próximo al argumento	Integer
`Sqr(x)`	Real o Integer	el cuadrado del argumento	Real
`Sqrt(x)`	Real o Integer	la raíz cuadrada del argumento	Real
`Trunc(x)`	Real	la parte entera del argumento	Integer
`Pi`	no posee	3.1415926535897932385	Real

- **Algunas funciones de cadenas de caracteres**

Nombre	Tipo(s) argumento(s)	Resultado
`Length(S)`	Cadena	Longitud lógica de S
`Pos(Substr, S)`	Cadenas	Posición donde comienza *Substr* en *S*. Si no existe retorna 0. Búsqueda *case-sensitive*.
`Copy(S, p, n)`	S: cadena ; p y n: enteros	Cadena de longitud *n* extraída de *S* a partir de la posición *p*.
`Trim(S)`	Cadena	Cadena sin blancos iniciales ni finales. Agregar *SysUtils* en sección *Uses*.

EJERCITACIÓN

Indicación: Los ejercicios 1 a 4, deben hacerse en **papel**

1.- Para los siguientes incisos codificar la parte declarativa de un programa eligiendo identificadores y valores de constantes e identificadores y tipos de variables.

a. variable para almacenar la altura de una persona
b. variable para almacenar la nota de un examen (0..100)
c. variable para almacenar la cantidad de alumnos de una escuela
d. variable/s para almacenar una fecha (ej. 24/12/2016)
e. variable para almacenar el importe de un deposito a interés en un banco
f. constante que represente el interés que paga un banco del 25%

2.- De acuerdo a las declaraciones efectuadas en el ejercicio 1, escribir las sentencias que corresponden a las siguientes acciones:

a. Asignar 1.80 a la variable que definió para la altura
b. Leer la nota del examen que ingresa por teclado
c. Escribir la cantidad de alumnos de una escuela
d. Asignar los datos correspondientes a una fecha
e. Calcular y escribir el monto obtenido en concepto de interés a partir del importe de un depósito y del interés que paga el banco

3.- Dada la siguiente definición de constantes y variables, realizar la prueba de escritorio con las sentencias de asignación propuestas:

```
Const
    K = 4;
Var
    X, Y, Z: Real;
    N, M, P: Integer;
Begin
   X := 4.8;
   Y := 2.4;
   Z := X/Y;
   N := 2;
   P := 10;
   M := P Mod K;
   X := X / K;
   N := N + 1;
End.
```

4.- Analizar los siguientes fragmentos de código y completando las líneas de puntos explicar qué proceso realizan. Agregar la sección `Var` con la declaración de variables.

```
a. writeln('Ingrese el radio de un círculo');
   readln(Radio);
   P := Radio *  ..........;
   S := Radio *......... * ..........;
   writeln('El perímetro del circulo es ', ..................... ,
            ' y la   ...................... es ', S);

b.  writeln('Escriba las coordenadas de los puntos P1 y P2');
    readln(X1, Y1);
    readln(X2, Y2);
    DIST := sqrt( sqr(X2 - X1) + sqr(Y2 - Y1)) ;
    writeln('Las coordenadas de P1 y P2 son', ................ , 'y', ....................);
    writeln('La distancia de P1 a P2 es ', .....................);
```

5.- Desarrollar los programas correspondientes para los siguientes problemas (utilizar sólo las variables necesarias) y realizar muchas pruebas de escritorio con distintos valores de variables:

a. Leer dos números enteros y calcular la suma, la diferencia, el producto y el cociente de ambos números. Luego, el programa debe mostrar los resultados en pantalla.
b. Calcular la distancia recorrida por un automóvil a velocidad constante en un tiempo determinado (ingresar la velocidad en km/h y el tiempo en horas).
c. Realizar un programa que convierta grados Celsius a Fahrenheit.
d. Resolver ahora la conversión inversa: de grados Fahrenheit a grados Celsius.
e. A partir de un importe en pesos y de lo que cotiza un real, dólar, un euro y una libra esterlina, calcular y mostrar cuánto se puede comprar de cada una de las divisas. Utiliza constantes para las cotizaciones de las distintas divisas.
f. Ingresar el peso y la altura de una persona, calcular y mostrar el IMC (el índice de masa corporal se calcula como peso / altura^2, estando la altura expresada en *metros*).
g. Leer tres números reales. Calcular y mostrar el promedio de los tres y la diferencia de cada uno de ellos con respecto al promedio.
h. A partir de un intervalo de tiempo dentro de un mismo día, determinado por un horario de inicio y otro de fin (ambos en horas, minutos y segundos), calcular y mostrar cuántos segundos transcurrieron. ¿Qué cambiaría si se pidiera cuántos minutos y segundos transcurrieron? Recuerda realizar muchas pruebas contemplando distintos valores de horas, minutos y segundos tanto en la hora de inicio como en la hora de fin.